

Project Situated AI Assignment 2:

Robotics and Reinforcement Learning

In this exercise you will utilise your knowledge from the courses 'Robotics' and 'Reinforcement Learning' to apply and evaluate deep reinforcement learning methods to robotics tasks.

In the following scenarios you will enable a simulated robot to control its body and perform simple movement tasks, which can later be used as primitives to solve more complex challenges.

Setup :

- As a starting point we provide you with a Google Colab notebook which is already able to run a baseline version of the task, although the robot can not yet solve it.
 - `quickstart_robotics_RL.ipynb`
 - The notebook contains code to install dependencies, generate two python files to improve these dependencies, and finally to initialise an RL agent within the simulated robot environment.
 - Use the quickstart to either create your own notebook or clean it up after understanding the basics of the assignment.
- We recommend you run this in Google Colab, however, it can be run locally or on kaggle if you prefer. The task can be computationally heavy, with a GPU providing significant speedup. Whatever platform you choose, be mindful of not losing work due to timeouts or incorrect logging.
- You will use two frameworks:
 - [Gymnasium Robotics](#)
 - [RL Baselines3 Zoo](#)

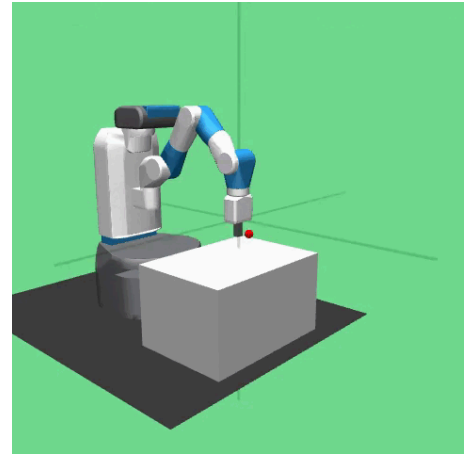
Exercise 1 :

Your first task to solve is the Fetch reaching task :

<https://robotics.farama.org/envs/fetch/reach/>.

Description :

The task in the environment is for a manipulator to move the end effector to a randomly selected position in the robot's workspace. The robot is a *7-DoF Fetch Mobile Manipulator* with a two-fingered parallel gripper. The robot is controlled by small displacements of the gripper in Cartesian coordinates and the inverse kinematics are computed internally by the MuJoCo framework.



Start this exercise using the environment '**FetchReachDense-v4**'.

Task 1.1 :

The first algorithm you will be applying to this task is the [Deep Deterministic Policy Gradient](#). DDPG is an extension of DQN, which is suitable for continuous action spaces. The Colab notebook demonstrates a basic setup of how to apply the task but does not learn anything yet.

Experiment with the hyperparameters such that the robot solves the task.

Tip : Record a video of the final policy to visually evaluate if the robot can solve the task.

Task 1.2 :

RL Baselines3 Zoo provides several RL algorithms. Select at least 3 of these (A2C, CrossQ, DDPG, PPO, SAC, TD3, TQC, or TRPO), determine reasonable hyperparameters which allow the algorithms to solve the task, and evaluate them using the same number of timesteps.

Utilize the logging features of *RL Baselines3 Zoo* to log the performance of the algorithms during training and use this data to create a graph showing a comparison of the performance of all selected algorithms over time. Run for each algorithm enough random seeds that you can plot the mean and standard deviation.

Task 1.3 :

Repeat the previous task, now using the environment '**FetchReach-v4**' (as opposed to *FetchReachDense-v4*). **What do you observe?** Do any of the algorithms solve the task? Read the documentation of the task and explain a possible reason for the observed behaviour.

Task 1.4 :

RL Baselines3 Zoo gives the possibility to combine an RL algorithm with Hindsight Experience Replay (*HER*). Research and explain what HER does, and combine it with at least two of the selected algorithms and rerun your experiments. **Explain what you observe and explain the reason for the change observed in the behaviour.**

Task 1.5 :

Go to file

```
Gymnasium-Robotics/gymnasium_robotics/envs/assets/fetch/robot.xml
```

and change the line

```
<inertial diaginertia="0.0086 0.0084 0.002" mass="2.1299" pos="0.1279  
0.0073 0" quat="0.4332 0.4332 0.5589 0.5589"></inertial>
```

to

```
<inertial diaginertia="0.0086 0.0084 0.002" mass="1.1299" pos="0.1279  
0.0073 0" quat="0.4332 0.4332 0.5589 0.5589"></inertial>
```

Apply your learned policy from task 1.1.

Do you notice a difference? Why or why not does the behavior of the policy change?

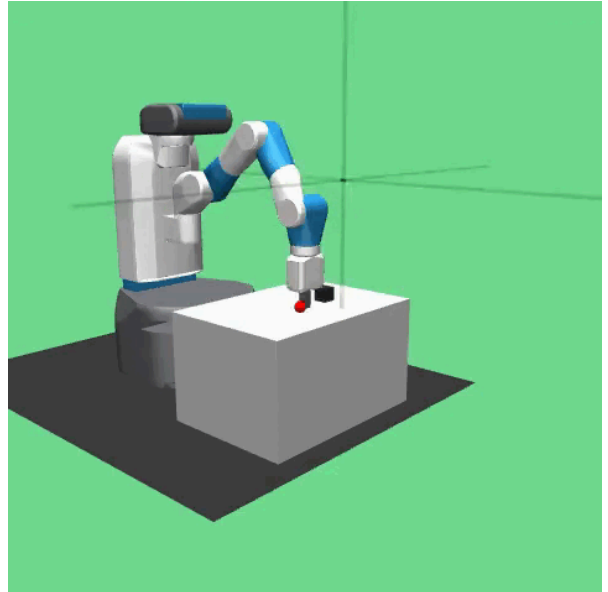
Tip : Remember to revert the change once you are done with this task. Also you must install the gymnasium-robotics package with `pip install -e`, or the change will not affect the program.

Exercise 2:

Select one reinforcement learning algorithm and solve for the task '**FetchPushDense-v4**' or '**FetchPush-v4**'.

<https://robotics.farama.org/envs/fetch/push/>

The task in the environment is for a manipulator to move a block to a target position on top of a table by pushing with its gripper. The robot is a 7-DoF Fetch Mobile Manipulator with a two-fingered parallel gripper. The robot is controlled by small displacements of the gripper in Cartesian coordinates and the inverse kinematics are computed internally by the MuJoCo framework. The gripper is locked in a closed configuration in order to perform the push task.



Report the process of solving the task.

This task requires significantly more training time compared to the previous task (~1e6 timesteps). Simple parameter search is unlikely to get you there. If you get stuck you can alternatively propose and implement some way to facilitate training. Think of changes in observations, reward, action space, or curriculum learning. Doing both the report of solving the task and these changes could net you a bonus point.

Exercise 3:

Select any other environment(s) from the Gymnasium-robotics suite and compare two reinforcement algorithms of your choice on it: <https://robotics.farama.org/> Comparing changes as in exercise 2 in observations, reward, action space, or curriculum learning is also allowed.

For this task, you are expected to **write a short report (max 4 pages)** where you:

- Hypothesise about the difference between the two algorithms.
- Describe your experimental setup.
- Report your results and conclusions.